

Air Quality Intelligence System

Advanced Database Management Systems — Course Report

Péter Dobosi

May 19, 2026

1 Team Information

Field	Value
Student	Péter Dobosi
Neptun code	DOP3BP
Course	Advanced Database Management Systems
Semester	2025/26 Spring
Repository	https://github.com/dobosipeter/advanced-data-warehouse-and-database-management-s
Live system	https://mw79on-demo.online
API docs	https://api.mw79on-demo.online/docs

2 System Architecture

The system is a real-time air quality monitoring and alerting platform ingesting data from the OpenAQ public API into a PostgreSQL database, processing it through triggers and transactions, and exposing it via a FastAPI REST backend and Streamlit GUI.

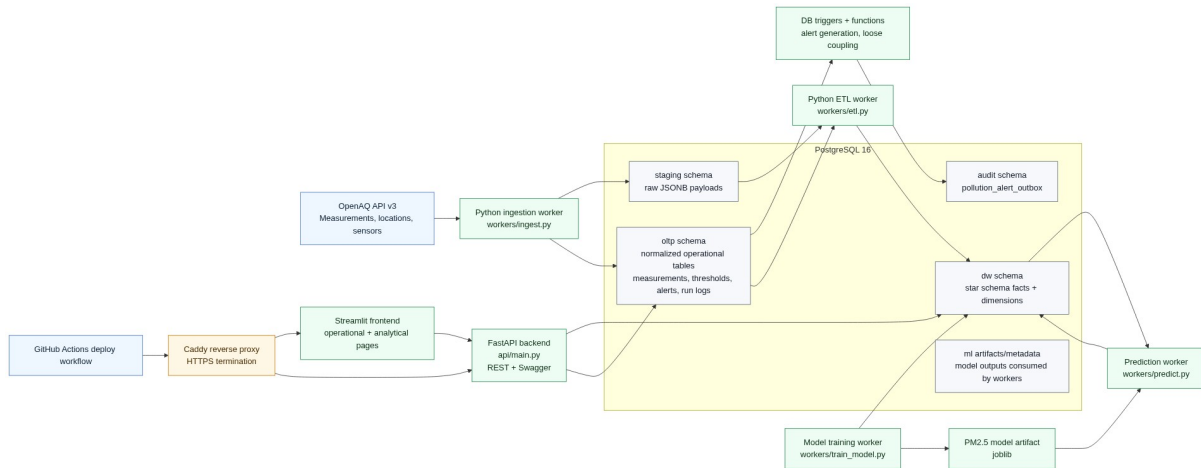


Figure 1: System architecture overview (emphasis on OLTP layer).

Key DBMS-oriented components:

- **PostgreSQL 16** — OLTP engine with schemas: `oltp`, `staging`, `audit`.
- **Triggers** — automatic alert generation on measurement insert; outbox event enqueue on alert state change.

- **Transactions** — all ingestion batches run inside explicit transactions with savepoints; failures roll back atomically.
- **Constraints** — CHECK, UNIQUE, FOREIGN KEY, and cross-column constraints on every table.
- **Partitioning** — `measurement_raw` is range-partitioned by `measured_at` (monthly).
- **Indexing** — B-tree indexes on foreign keys, composite indexes on hot query paths.
- **Backup** — scripted `pg_dump` with rotation and optional off-site upload.
- **FastAPI** — RESTful API with OpenAPI/Swagger auto-docs.
- **Streamlit** — Operational GUI for threshold management, alert review, and system monitoring.

3 Functional Specification

The operational (DBMS) layer provides the following features:

1. **Data Ingestion** — Incremental and initial bulk load from OpenAQ v3 API via Python worker.
2. **Threshold Management** — CRUD for per-city, per-pollutant alert rules with severity levels (low / moderate / high / critical).
3. **Automatic Alert Generation** — PostgreSQL trigger fires on every new measurement, matches against active threshold rules, and inserts an alert row.
4. **Outbox Pattern** — A second trigger enqueues alert lifecycle events into an audit outbox table for downstream consumers.
5. **Alert Review Workflow** — Operators can transition alerts through open → reviewed → closed with notes.
6. **Transaction Safety** — Ingestion uses explicit transactions; partial failures are rolled back without corrupting existing data.
7. **Partitioning** — Monthly range partitions on the measurement table enable partition pruning on time-range queries and simplify archival.
8. **Indexing** — Targeted indexes accelerate JOIN-heavy API queries (measurements by sensor, alerts by status, thresholds by city/parameter).
9. **Backup & Maintenance** — Cron-driven `pg_dump` with configurable retention and VACUUM/ANALYZE scheduling.
10. **REST API** — FastAPI backend exposes health, locations, measurements, alerts, thresholds, ingestion runs, and predictions.
11. **Operational GUI** — Streamlit dashboard with pages for station exploration, threshold editing, alert management, and system status.
12. **CI/CD Deployment** — GitHub Actions workflow deploys changes to the Hetzner VM automatically on push to `main`.

4 Logical Data Model (OLTP)

The OLTP schema (`oltp`) contains the following tables:

Table	Primary Key	Purpose
ingestion_run_log	ingestion_run_id	Track each pipeline run (status, counts, errors)
parameter	parameter_id	Pollutant definitions (pm25, pm10, no2, o3, ...)
location	location_id	Monitoring stations with coordinates and metadata
sensor	sensor_id	Physical sensors linking locations to parameters
measurement_raw	(measurement_id, measured_at)	Raw observations (partitioned by month)
threshold_rule	threshold_rule_id	Alert threshold definitions per city/pollutant/level
pollution_alert	pollution_alert_id	Generated alerts referencing measurements & rules

Table 1: OLTP tables.

Supporting schemas:

- `staging.raw_api_response` — stores raw JSON API responses for auditability.
- `audit.pollution_alert_outbox` — event sourcing table for alert lifecycle events.

Key Relationships

- `sensor` → `location` (many-to-one)
- `sensor` → `parameter` (many-to-one)
- `measurement_raw` → `sensor` (many-to-one)
- `measurement_raw` → `ingestion_run_log` (many-to-one, nullable)
- `pollution_alert` → `measurement_raw` (composite FK)
- `pollution_alert` → `threshold_rule` (many-to-one)

5 Technologies Used

Technology	Role
PostgreSQL 16	RDBMS (OLTP + DW + audit schemas)
Python 3.12	Worker scripts, API, frontend
FastAPI	REST API with automatic OpenAPI docs
Streamlit	Operational + analytical dashboards
Docker Compose	Container orchestration
Caddy	Reverse proxy with automatic HTTPS
GitHub Actions	CI/CD pipeline (SSH deploy)
Hetzner Cloud	Production VM (Ubuntu 24.04)

Table 2: Technology stack.

Notable PostgreSQL Features Demonstrated

- **PL/pgSQL triggers** (alert generation, outbox enqueue)
- **Range partitioning** (monthly measurement partitions)
- **CHECK constraints** with cross-column validation
- **Composite primary/foreign keys**
- **JSONB columns** for flexible raw payload storage
- **Explicit transactions with savepoints** in ingestion scripts
- **pg_dump** for automated backup and recovery